COMPUTATIONAL BIOLOGY · MODULE 11

Working with Biological Databases and File Formats

Where biological data lives and what shape it arrives in — the major public databases, the file formats that carry sequence and annotation data, and how to retrieve the exact record needed in a way that can be traced and reproduced.

AUTHOR	Prachya Biswas
PROGRAMME	Computer Science, BRAC University
TOPIC	Biological Databases & File Formats
SERIES	Computational Biology Meets Cells — Module 11

Contents

1. Why Databases and Formats Come First

2. Primary Sequence Databases

3. Protein Databases — UniProt

4. Structural Databases — The Protein Data Bank

5. Specialized Databases — Pathway, Disease, Variant

6. FASTA and FASTQ

7. Quality Scores and Metadata

8. GFF and GTF Annotation Formats

9. Genome Browsers — UCSC and Ensembl

10. Data Retrieval Strategies and Batch Downloads

11. Data Curation, Versioning, and Reproducibility

12. Hands-on — Searching and Retrieving a Sequence Record

13. Key Takeaways

14. What's Next

1. Why Databases and Formats Come First

Every analysis starts with two unglamorous questions: *where do I get the data*, and *what form is it in*. Getting either one wrong does not usually produce an error message. It produces a result that looks fine and is quietly built on the wrong record, the wrong genome version, or unreliable bases.

The databases in this module form a ladder of increasing meaning:

Layer	Resource	What it adds
Raw sequence	GenBank, ENA, DDBJ	The letters themselves
Protein knowledge	UniProt	Name, function, domains, evidence
3D structure	PDB	The spatial arrangement of the molecule
Biological context	Pathway, disease, and variant databases	What process it belongs to and what a change means clinically

Each step up the ladder moves from "what is written" toward "what it means".

2. Primary Sequence Databases

Primary sequence databases are the foundation of sequence-based bioinformatics. They hold the DNA and RNA sequences that researchers submit, and make them available to everyone.

Database	Maintained by	Region
GenBank	NCBI	United States
ENA (historically the EMBL database)	EMBL-EBI	Europe
DDBJ	DNA Data Bank of Japan	Japan

They are synchronized. The three exchange data routinely, so they behave like three branches of one library holding the same shared collection. A sequence submitted to any one of them becomes available through all three. This collaboration is known as the INSDC.

"Primary" means submitter-owned. These archives store what researchers deposited, largely as deposited. That has two consequences worth remembering:

- The same gene may appear many times, submitted by different groups.
- Quality and annotation depth vary from record to record.

Why open access matters. If sequence data sat in isolated silos, every lab would have to re-sequence what another lab already has, and nobody could check anyone else's result. Shared access is what enables reproducibility, comparison, analysis, and discovery. These databases are the starting point for almost everything downstream: analysis, annotation, and comparison.

3. Protein Databases — UniProt

Protein databases focus on the translated product, not just the raw nucleotide record. **UniProt** is the central one. It connects a protein sequence to its name, function, domains, and wider biological context.

A sequence without annotation has limited value. A string of amino acids on its own says almost nothing. The value is in what is attached to it: what the protein does, where in the cell it acts, which domains it contains, and what evidence supports each claim.

UniProt has two sections, and the difference matters every time a record is used:

Section	How entries are produced	How far to trust the annotation
Swiss-Prot (reviewed)	Manually curated by experts from the literature	High
TrEMBL (unreviewed)	Generated automatically by computational pipelines	Treat as a prediction

Similar sequence does not guarantee identical function. Two proteins can look alike and still do different jobs. This is exactly why curated evidence matters: a function inferred only from similarity is a hypothesis, and a function demonstrated experimentally is a fact.

How researchers use it. To infer the function of a newly found protein, and to compare proteins across organisms.

> **Key idea:** protein databases are interpretation tools. UniProt turns sequence data into functional biological insight.

4. Structural Databases — The Protein Data Bank

The **Protein Data Bank (PDB)** is the main public database for three-dimensional structures of proteins, DNA, and RNA.

The division of labour with UniProt is clean:

- **UniProt** gives identity and function — *what is this molecule and what does it do?*
- **PDB** gives the spatial arrangement — *what does it physically look like?*

Structure often explains what sequence cannot. A mutation that looks harmless in the sequence may sit exactly in the pocket where a drug or substrate binds. Only the 3D structure shows that. So the deeper question is not just "what molecule is this?" but "how does its architecture drive its function?".

Where structures come from:

Method	In brief
X-ray crystallography	The molecule is crystallised and X-ray diffraction is used to work out atom positions
NMR spectroscopy	Structure is determined in solution; suited to smaller proteins
Cryo-electron microscopy	Molecules are flash-frozen and imaged; suited to large complexes

What a PDB entry contains:

- **Coordinates** — the x, y, z position of every atom.
- **Chains** — the separate polymer chains in the structure.
- **Ligands** — bound small molecules, such as a drug or cofactor.
- **Method metadata** — how the structure was determined, including its resolution.

What it is used for. Studying binding and active sites, seeing where a mutation lands, comparing related structures, and designing drugs.

5. Specialized Databases — Pathway, Disease, Variant

Sequence and structure tell you what a molecule is. Specialized databases add the context around it.

Type	The question it answers	Examples
Pathway	If this gene matters, which process does it affect?	KEGG, Reactome
Disease	Is this gene or mutation linked to a clinical condition?	OMIM
Variant	Has this exact change been seen before, and what significance was reported?	ClinVar, dbSNP, gnomAD

Pathway databases place a gene or protein inside a larger biological process. A gene rarely acts alone; its importance usually comes from the pathway it sits in.

Disease databases link genes and mutations to clinical conditions.

Variant repositories answer the most practical question in clinical genomics. When a change is found in a patient's sequence, the first thing to check is whether it is *known* or *unknown*. A variant that is common in healthy people is very unlikely to cause a rare disease. A variant previously reported as disease-causing is a strong lead.

The three are connected as a triangle: **gene — variant — disease**. A finding becomes meaningful only when all three corners are filled in.

> **Key idea:** these resources turn raw sequence findings into biological and clinical meaning.

6. FASTA and FASTQ

FASTA and FASTQ are the standard formats for moving sequence data between tools.

FASTA

The simplest format there is: a header line beginning with >, followed by the sequence.

```
>seq1 example DNA sequence
ATGCGTACGGTAACTGA
```

It works for DNA, RNA, or protein. Use FASTA when the main focus is **the sequence itself**: reference genomes, gene sequences, protein sequences.

FASTQ

FASTQ stores the sequence **and** a quality score for every base. Each read takes exactly four lines:

```
@read1
ATCGATCG
+
!!!??@<=
```

Line	Content
1	@ followed by the read identifier
2	The sequence
3	+ (a separator)
4	The quality string — one character per base, same length as line 2

FASTQ is what a sequencing machine produces. The quality scores show how reliable each base call is.

Choosing between them

	FASTA	FASTQ
Contains	Sequence only	Sequence plus per-base quality
Typical source	Databases, assembled references	Raw sequencer output
Typical use	Reference, alignment target, search	Quality control, trimming, read alignment

> **Ask first:** is this sequence only, or sequence plus evidence of how reliable it is? The answer decides the next analysis step.

7. Quality Scores and Metadata

Quality scores

Every base in a FASTQ file has a **Phred quality score** `Q`, which expresses the estimated probability `P` that the base call is wrong:

$Q = -10 \times \log_{10}(P)$

Q	Error probability	Accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1,000	99.9%
40	1 in 10,000	99.99%

How the score is stored. Each score is written as a single character. In the standard encoding (Phred+33), the character's ASCII code minus 33 gives `Q`.

Worked example, decoding the quality string `!!!??@<=`:

Character	ASCII code	Q = code – 33	Reading
!	33	0	No confidence at all
?	63	30	99.9% accurate
@	64	31	Slightly better than Q30
<	60	27	Good
=	61	28	Good

So in this read the first three bases are unusable, and the rest are reliable. The same decoding in Python:

```
quality = "!!!??@<="
scores = [ord(ch) - 33 for ch in quality]
print(scores)   # [0, 0, 0, 30, 30, 31, 27, 28]
```

Why it matters. Low-quality bases can distort alignments, assemblies, and variant calls. A sequencing error can look exactly like a real mutation. Poor input data misleads everything downstream, which is why quality control is the first step of nearly every pipeline.

Metadata

Metadata is the information *about* the data: which sample, which sequencing run, which instrument, which labels and conditions.

Picture two tubes on a bench. One is labelled with a sample ID, run, and instrument. The other just says "Stuff". The contents might be identical, but only the first one can be used in an experiment. Sequence data without metadata is the second tube.

> **Key idea:** quality scores show reliability; metadata shows biological meaning. One tells you whether to trust the bases, the other tells you what those bases represent in the real experiment.

8. GFF and GTF Annotation Formats

A genome sequence on its own is billions of letters with no labels. **Annotation** files add the labels: where the genes are, where each exon starts and ends, which regions code for protein, and which transcripts exist. They convert raw sequence into interpretable biology.

Annotation tells you three things about a feature: **what** it is, **where** it is, and **what it is related to**.

The shared structure

Both GFF and GTF are plain text with one feature per line and nine tab-separated columns:

#	Column	Meaning
1	seqname	Chromosome or contig
2	source	Where the annotation came from
3	feature	Type: gene, transcript, exon, CDS, ...
4	start	Start position (1-based)
5	end	End position (inclusive)
6	score	A numeric score, or `.` if none
7	strand	`+` or `-`
8	phase / frame	Reading-frame offset for CDS features, or `.`
9	attributes	Identifiers and relationships

Where they differ

Columns 1 to 8 are essentially the same. The difference is in column 9, which is how features are linked to each other.

GFF3 uses `key=value` pairs and builds a hierarchy with `ID` and `Parent`:

```
chr17  HAVANA  gene  43044295  43125364  .  -  .  ID=gene:ENSG00000012048;Name=BRCA1
chr17  HAVANA  mRNA  43044295  43125364  .  -  .
    ID=tx:ENST00000357654;Parent=gene:ENSG00000012048
chr17  HAVANA  exon  43124017  43124115  .  -  .  Parent=tx:ENST00000357654
```

GTF uses `key "value";` pairs and repeats `gene_id` and `transcript_id` on every line:

```
chr17  HAVANA  exon  43124017  43124115  .  -  .  gene_id "ENSG00000012048"; transcript_id
    "ENST00000357654"; gene_name "BRCA1";
```

(Lines are illustrative; real files separate columns with tabs.)

	GFF3	GTF
Attribute syntax	`key=value;`	`key "value";`
Linking features	`ID` and `Parent` build an explicit tree	Shared `gene_id` and `transcript_id`
Typical use	General-purpose genome annotation	Gene-expression (RNA-seq) pipelines

Why the small difference matters. Tools are strict about which format they accept. Hand a GFF3 file to a tool expecting GTF and it may fail, or it may run and silently assign nothing to any gene. The format choice affects tools, analysis, RNA-seq quantification, and visualisation, so it is worth checking which one a tool expects before starting.

9. Genome Browsers — UCSC and Ensembl

A genome browser displays annotations in their genomic context, as a picture. The **UCSC Genome Browser** and **Ensembl** are the two main ones. Both let you search for a gene or region, zoom in and out, and inspect what is there.

What you look at: exons, transcripts, variants, and neighbouring features, stacked as horizontal **tracks** along the chromosome.

Why context matters. A variant is not interpreted in isolation. Whether it falls inside an exon, in an intron, or in a regulatory element next to the gene changes what it is likely to do. Nearby transcripts and regulatory elements all affect the interpretation.

The single most important setting: the assembly. A genome assembly is a specific version of the reference genome. Coordinates are only meaningful together with the assembly they refer to.

Assembly	Also called	BRCA1 location (approx.)
GRCh37	hg19	chr17:41,196,312–41,277,500
GRCh38	hg38	chr17:43,044,295–43,125,364

Same gene, same chromosome, but the coordinates differ by almost two million bases. Mixing assemblies between a variant file and an annotation file is one of the most common silent errors in genomics.

> **Key skill:** orient yourself, pick the right assembly, and read the annotations carefully.

10. Data Retrieval Strategies and Batch Downloads

Downloading records one at a time through a web page works for one gene. It fails for a hundred genes, a set of regions, or several datasets: it is slow, error-prone, and impossible to repeat exactly.

A retrieval plan, in order:

1. **Define exactly what is needed.** Sequences, annotations, variants, or coordinates? For which organism and which assembly?
2. **Choose the right database** for that data type.
3. **Choose the retrieval type:**

Type	When to use it
Single	One specific record, looked up by its identifier
Filtered	Everything matching a set of criteria (organism, gene, date, record type)
Batch	Many records at once, from a list of IDs, a filter, or a preset dataset

4. **Record how it was done**, so it can be repeated.

Common tools for batch work include NCBI's Entrez utilities, Ensembl BioMart, and the UCSC Table Browser.

Why plan early. Retrieval is not "download more files". It is retrieving the *right* data in the *right* way from the start. Feeding wrong data into a pipeline does not stop the pipeline; it just produces wrong results that take much longer to notice.

> **Key idea:** efficient retrieval improves speed, reproducibility, organisation, and confidence in the result.

11. Data Curation, Versioning, and Reproducibility

After retrieval, data has to stay **trustworthy, traceable, and reusable**. Without that, nobody can say where a file came from or why a result changed between two runs.

Data curation means making sure a dataset is accurate and complete, with clear labels, proper metadata, and no duplicates.

The test: *could a teammate understand this dataset months from now without asking me anything?*

Versioning means recording exactly which version of a file or database was used. Databases are updated continuously: records are corrected, annotations change, new releases come out. The same query run six months later can return something different.

Sequence records carry their version in the identifier itself:

Identifier	Meaning
`NM_007294`	The accession: the record in general
`NM_007294.4`	Accession **and** version: this exact sequence

Always record and cite the versioned form.

Reproducibility requires three things to be written down:

1. **Sources** — which database, which record identifiers.
2. **Versions** — database release, accession versions, genome assembly, and the date of retrieval.
3. **Processing decisions** — what was filtered, removed, or changed, and why.

A minimal record to keep alongside any downloaded dataset:

```
Source:      NCBI Nucleotide
Record:      NM_007294.4
Query:       BRCA1[Gene] AND "Homo sapiens"[Organism]
Retrieved:   2026-11-15
Assembly:    GRCh38
Processing:  none (record used as downloaded)
```

In research, clinics, and industry these practices are not administrative decoration. They are what allows a result to be checked, and a result that cannot be checked cannot be trusted.

12. Hands-on — Searching and Retrieving a Sequence Record

The goal: search a database, confirm it is the correct record, and retrieve it with full traceability.

Treat retrieval like citing the exact edition of a paper, not a similar-looking one. Before using a record, verify three things:

- **Organism** — is it the right species?
- **Isoform** — is it the right transcript or protein variant of the gene?
- **Version** — is it the version intended?

A wrong record does not announce itself. It quietly breaks primer design, alignment, and everything after.

The script

This uses Biopython's `Entrez` module to talk to NCBI. It needs Biopython (`pip install biopython`) and an internet connection.

```
from Bio import Entrez
from Bio import SeqIO


def search_and_retrieve_sequence(email, search_term, database='nucleotide',
                                 rettype='gb', retmode='text'):
    # Always tell NCBI who you are
    Entrez.email = email
```

```
    # Step 1: search the database
    print(f"Searching for '{search_term}' in the {database} database...")
    handle = Entrez.esearch(db=database, term=search_term, retmax=1)
    record = Entrez.read(handle)
    handle.close()

    # Step 2: get the list of matching IDs
    id_list = record['IdList']
    if not id_list:
        print("No records found.")
        return None

    # Step 3: fetch the full record for the first ID
    print(f"Fetching record with ID: {id_list[0]}")
    handle = Entrez.efetch(db=database, id=id_list[0],
                           rettype=rettype, retmode=retmode)
    sequence_record = SeqIO.read(handle, rettype)
    handle.close()

    # Step 4: display the details needed for verification
    print("\nSequence Record Details:")
    print(f"ID: {sequence_record.id}")
    print(f"Name: {sequence_record.name}")
    print(f"Description: {sequence_record.description}")
    print(f"Annotations: {sequence_record.annotations}")
    print(f"Sequence: {sequence_record.seq[:50]}... (truncated)")

    return sequence_record


sequence_record = search_and_retrieve_sequence(
    email='your_email@example.com',
    search_term='BRCA1[Gene] Homo sapiens'
)
```

Walkthrough

Step	Code	What happens
Setup	`Entrez.email = email`	NCBI asks every user to identify themselves, so they can make contact before blocking a script that misbehaves
Search	`Entrez.esearch(...)`	Runs the query and returns a list of matching record IDs — not the records themselves
Parse	`Entrez.read(handle)`	Converts NCBI's XML reply into a Python dictionary
Guard	`if not id_list:`	If nothing matched, stop cleanly instead of crashing on `id_list[0]`
Fetch	`Entrez.efetch(...)`	Downloads the full record for one ID
Parse	`SeqIO.read(handle, rettype)`	Turns the GenBank text into a `SeqRecord` object
Verify	the `print` block	Shows ID, name, description, and annotations so the record can be checked by eye
Return	`return sequence_record`	Hands the record back for later analysis

The pattern to remember is **search → get IDs → fetch → verify**. `esearch` finds, `efetch` retrieves. They are two separate requests.

Parameters

Parameter	Value used	Meaning
`database`	`'nucleotide'`	Which NCBI database to search
`rettype`	`'gb'`	Return the full GenBank record, including annotations
`retmode`	`'text'`	Return plain text
`retmax`	`1`	Return at most one ID
`search_term`	`'BRCA1[Gene] Homo sapiens'`	`[Gene]` restricts `BRCA1` to the gene-name field

What the printed details are for

The print block is the part that serves the real lesson. `sequence_record.id` shows the versioned accession. `description` shows the organism, gene, and transcript variant. `annotations` holds the organism, taxonomy, and record dates. Together they answer the organism / isoform / version checklist. The sequence is cut to the first 50 bases simply because the full sequence would flood the screen.

Weak points worth knowing

- `retmax=1` **takes whatever comes first.** The first hit for a broad query is not guaranteed to be the record wanted. It could be a different transcript variant or a much larger genomic record. The script retrieves *a* BRCA1-related record, not necessarily *the* intended one. This is exactly why the verification step exists.
- **The query is loose.** `Homo sapiens` is not tied to the organism field. A tighter query is `BRCA1[Gene] AND "Homo sapiens"[Organism]`.
- `SeqIO.read(handle, rettype)` **reuses** `rettype` **as the parser format.** That works for `'gb'` and `'fasta'` because those happen to be valid format names for both NCBI and `SeqIO`. It is a coincidence, not a rule, and other return types will break it.
- **No error handling.** A dropped connection or an invalid reply raises an exception.
- **Rate limits.** NCBI allows about 3 requests per second without an API key. A loop over many genes needs a short pause between requests.

A more traceable version

When the exact record is already known, fetching it directly by its versioned accession removes the guesswork:

```
from Bio import Entrez, SeqIO

Entrez.email = "your_email@example.com"

try:
    handle = Entrez.efetch(db="nucleotide", id="NM_007294.4",
                           rettype="gb", retmode="text")
    record = SeqIO.read(handle, "genbank")
    handle.close()
except Exception as error:
    print(f"Retrieval failed: {error}")
else:
    print(record.id)                          # versioned accession
    print(record.annotations["organism"])     # verify the organism
    print(record.description)                 # verify the transcript variant
    print(len(record.seq))                    # sanity-check the length
```

> **Habit to build:** search carefully, verify deliberately, retrieve precisely.

13. Key Takeaways

- GenBank, ENA, and DDBJ hold the same shared, submitter-provided nucleotide collection.
- UniProt adds function to sequence; reviewed (Swiss-Prot) and unreviewed (TrEMBL) entries deserve different levels of trust.
- PDB adds 3D structure, which often explains what sequence alone cannot.
- Pathway, disease, and variant databases supply the biological and clinical context.
- FASTA is sequence only; FASTQ is sequence plus a quality score per base.
- Phred Q30 means a 1 in 1,000 chance that a base is wrong.

- GFF and GTF share eight columns and differ in the ninth; tools care about the difference.
- Genomic coordinates mean nothing without the assembly they belong to.
- Record the source, the version, and every processing decision, or the result cannot be reproduced.
- Retrieval is a four-step habit: search, get IDs, fetch, verify.

14. What's Next

With the data sources and formats understood, the natural next step is doing something with the sequences themselves: comparing them. That means sequence alignment — how two sequences are lined up, how similarity is scored, and how a query is searched against an entire database.